

Stat 5810/6810 — Project 2 (90 Points) — Tu 11/20/2012

General Instructions

Due: Fr 12/7/2012, 11:59pm (by e-mail)

For this project, you will work with XML files from Apple's iTunes Music Library, called `iTunes Music Library.xml`. The format of this XML file is further described at <http://www.xml.com/pub/a/2004/11/03/itunes.html>. The iTunes wiki page at <http://en.wikipedia.org/wiki/iTunes> may also provide links to useful references. Ultimately, google for keywords such as `iTunes Music Library.xml` for further information.

For some of the tasks, you have to work with the file that is based on my iPod usage during the summer months. I usually listen to music on an iPod while lawnmowing and during other major gardening activities.

Early in the summer, I have removed all previous music from my iPod and I have uploaded my 2012 set of songs (mostly 80ies and 90ies music this year) for this summer. I did not further add any new songs later on.

For some of the tasks, you also need to work with an alternative `iTunes Music Library.xml` file, either your own version or that from a friend. On a PC, this file usually is located in `DISK:\Documents and Settings\UserName\My Documents\My Music\iTunes`. You should copy the file, but not delete or rename the original file (unless you want to erase your past music history).

The XML R documentation at <http://cran.r-project.org/web/packages/XML/XML.pdf> refers to Apple's iTunes files a few times. Of particular relevance is the command `getSibling`.

You are allowed to work in groups of up to two students. A final report in $\text{\LaTeX}$ with your figures, a summary of your results, main features of your R code you would like to highlight, and the R code itself in the appendix (with the functions and the specific commands that are needed to produce the results outlined below) must be turned in as a .pdf file. I also need the .R file and your iTunes Music Library.xml file! Your R code must produce reproducible results. This means when I rerun your R code with the same input data, I will get exactly the same results as in your report.

Here is the list of tasks for this project:

1. First open the `iTunes Music Library.xml` file in a text editor and try to understand its basic structure. How are data of interest stored and how does this file match and differ from the XML tree structure discussed in class?

Once you have a feeling for the basic structure, write a function that reads in the data from a file specified via `fname` and that creates an R data frame that has as many rows as there are song in the iTunes library. The columns should contain the data

that are related to each song, such as “Track ID”, “Name”, “Kind”, ..., “Library Folder Count”. Note that the number of data fields in the XML file may differ for each song, for example, “Play Count” and related fields only are added to the XML file if a song has been played at least once. [In your main text, you will have to explain how you are addressing this issue.] Your function should have a second input parameter, called `SpecialFormat`, that is `FALSE` by default. However, if `TRUE`, you need to address the points from task 2. Finally, your function should return the resulting data frame.

2. My iTunes Music Library.xml file has a few unusual characteristics as the songs are self-transformed mp3s (from WAVs) where the “Name” is of the form

```
Artist - Title (Year)
Artist - Title (Other Info) (Year)
```

Using regular expressions, split the provided “Name” into three new variables, “Artist”, “Name2”, and “Year”. Then add these new variables to the data frame and remove the old “Name” (and rename “Name2” as “Name”). This is similar to our movie example. Check that the results are meaningful! There could be years (or other numbers) as part of the title, there could be more than one hyphen, or some information might be missing.

3. Assuming that there are N songs and M different artists in the XML file, write (and carefully test) the following functions:

```
most_played_songs(n, truncate = FALSE)
most_recent_songs(n, truncate = FALSE)
most_frequent_artist_in_file(m, truncate = FALSE)
```

Warnings should be produced if $n > N$ or $m > M$ — and the functions should use N or M , respectively, in such a case.

Note that part of this information is stored in the XML file, but this is incomplete as n is fixed. So, it would be best to ignore the existing information entirely and recreate this information from the data, but **it might be a very good idea to compare your results with the results that are part of the XML file**. Comment on the outcome of this comparison in your report!

As our data are count data, there likely will be ties, e.g., several songs played the same number of times, or the same number of songs for multiple artists. In the default `truncate = FALSE` setting, your functions will return all additional songs (or artists) that got the same number of plays. For example, if $n = 10$ and the 10th most-played song has been played 3 times and there are 5 more songs that have been played 3 times, all of these songs will also be returned.

However, if `truncate = TRUE`, then only the first songs/artists that are tied will be returned, according to some internal sorting. This could be alphabetical, by “Date Added”, or any other criterium you think that is meaningful. You should produce a warning that there are k additional songs/artists with a count that is identical to the n^{th} song/artist. But, this warning only needs to be produced if there are indeed additional songs/artists with the same count. Obviously, it could also be the case that the $(n + 1)^{\text{st}}$ song/artist has a lower count.

For the written report, apply these 3 functions with $n = m = 10$ and both options for `truncate` to both, my and your, `iTunes Music Library.xml` files, i.e., there should be $3 \times 2 \times 2 = 12$ function calls and their outputs be reported.

4. Write some functions that plot components of the data, e.g., a histogram of the “Year” variable and a bar chart that shows the number of songs that last were played on a particular day when at least one song has been played. Unfortunately, the XML file only contains “Play Date UTC” which is the last date and time a song has been played, but earlier dates do not get recorded. The bar chart should be sortable by date or in increasing or decreasing order of songs last played. Function names and any additional parameters are up to you.

For the written report, create the histogram for my `iTunes Music Library.xml` file and two bar charts (in meaningful orders) based on my and your `iTunes Music Library.xml` files.

5. Note that there is a “Location” variable. This indicates the directory from which a song originates. In fact, this is related to the compilation CD that originally hosted the songs in this directory. Write a function that returns a data frame with the “Location” information, the number of songs in this “Location”, the total time for all songs in this “Location”, and the average time for the songs in this “Location”. The `plyr` R package may be helpful here. Include the output of this function in your report.

Note that “Total Time” is not in seconds. Rather, you need to find the information yourself how time (in seconds) can be calculated from the numbers provided in the data. In your report, indicate the necessary transformation and the source (URL or any printed document) that indicates how to obtain time (in seconds). Likely, you will need to google for this transformation or read through several wiki pages.

To allow me to check your transformation, indicate the time (in seconds) for the shortest and the longest songs (including artist and title) in your report. If you can’t figure out how to translate “Total Time” into seconds, just work with the provided numbers.

6. You will notice, that in one of the locations, there are uniquely songs that contain the name of a US city. However, there are additional songs throughout the entire

data set that also contain the name of a US city. We want to find all songs that refer to a US city with a current population of more than 100,000. These should be full city names, but no abbreviations, such as L.A. (for Los Angeles) or NYC (for New York City).

You should consider that any combination of 1, 2, or 3 consecutive words in the title might be a city (think of Boston, Los Angeles, or Salt Lake City). Your task is to determine which songs contain a US city name, based on the list of 285 US cities provided at http://en.wikipedia.org/wiki/List_of_United_States_cities_by_population. We do not worry if a city name is matched with the wrong state, e.g., there is a Hollywood, FL, in this list while most people intuitively think of Hollywood as a part of Los Angeles, CA.

Your function should return a data frame that contains the city name, the artist, the song title, and the 2011 population estimate, based on the wiki page. The city names could be arranged in increasing or decreasing order of population, or in alphabetical order, depending on an input parameter. Again, all further details are left to you.

For the report, outline the algorithm, i.e., the your conceptual approach, how you extract the potential city names of variable lengths from the data and compare those to the names in the city wiki file. Be prepared that a song title may contain more than just one city name, such as in “Boulder To Birmingham” (http://en.wikipedia.org/wiki/Boulder_to_Birmingham).

Also include the results of this function when applied to my (sorted in decreasing order of population) and your (sorted alphabetically) iTunes Music Library.xml files. Be aware that some XML files may not contain any song with a city name — so your code should produce a warning in such a case (but it shouldn’t stop with an error message).

Please let me know whenever you have questions — and good luck!!!